SentinelAI: An Intelligent SOC Incident Triage Copilot

LLM application capstone - final project report

Model: gemma-4-E4B-it via the class OpenAI-compatible proxy · Results generated 2026-10-16 · Dataset eval/datasets/retrieval_eval_v1.jsonl (sha256/16 ef03063861a8e6be)

Abstract

SentinelAI assists junior Security Operations Center (SOC) analysts in triaging synthetic security alerts. It combines schema-validated LLM extraction with a literal grounding check, retrieval-augmented generation over 19 original fictional playbooks, a bounded tool-calling agent with three read-only tools, a deterministic severity matrix, and a citation-checked incident report served through FastAPI and a web dashboard. On a held-out test split of 16 labelled queries (14 answerable), the configuration selected on the development split (hybrid-sbert-section-a0.3) improved Recall@3 from 0.679 to 0.750 and MRR@5 from 0.696 to 0.729 over a BM25 fixed-chunk baseline. The gain is small and the test set is tiny; end-to-end LLM metrics are reported in Section 7.

1. Introduction and problem statement

Tier-1 analysts face high alert volumes in heterogeneous formats. Typical junior mistakes are (a) confusing similar-looking alerts that need different playbooks (a brute-force burst versus a successful login after that burst), (b) filling gaps with guesses, and (c) inconsistent severity. Generic chat-with-documents tools make (b) and (c) worse because a language model will confidently invent hostnames, citations and priorities. SentinelAI is designed around the opposite principle: the model handles language, while everything that must be reproducible or auditable is done by code and checked against evidence.

2. Intended users and objectives

- **Users:** junior SOC analysts and students learning incident triage; instructors reviewing the workflow.
- **O1** extract structured incident fields without inventing values (Pydantic + grounding check).
- **O2** retrieve the right playbook and cite only documents actually retrieved (RAG + citation validation).
- **O3** let an agent choose investigation tools safely (allow-list, schemas, explicit stop conditions).
- **O4** compute severity deterministically from a documented policy (PB-002) and an asset catalogue.
- **O5** measure retrieval quality against a baseline with a dev/test protocol and report it honestly.

3. System architecture and data flow

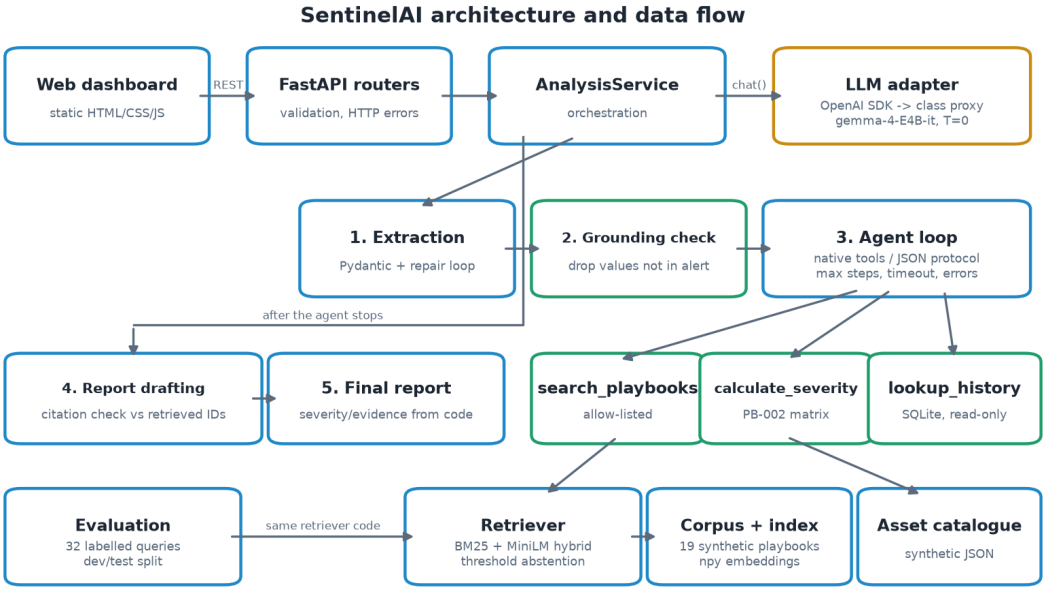


Figure 1. Layers and request flow. Green boxes are deterministic safety components.

Routers only validate HTTP input and map errors (422 invalid input, 400 malformed IDs, 502 failed model validation, 503 missing proxy configuration). *AnalysisService* orchestrates: extraction, grounding, agent loop, tool enforcement, report drafting and persistence. Repositories wrap SQLite (saved analyses and synthetic incident history, parameterised SQL only) and the JSON asset catalogue. A single LLM adapter is the only code that talks to the model, which allows the whole pipeline to be tested with a scripted fake client.

4. LLM integration, RAG, structured output and agent design

LLM integration. The official OpenAI Python SDK is pointed at the class proxy using OPENAI_API_KEY and OPENAI_BASE_URL from the environment; model gemma-4-E4B-it; temperature 0. JSON mode is requested and automatically dropped if the proxy rejects it. Proxy compatibility probe results: list_models=ok, chat=ok, system_role=ok, json_mode=ok, native_tools=ok

Structured output. Extraction and report drafts are validated by Pydantic models (enumerated incident types, IP validation, bounded lists). On failure the validator message is sent back and the model retries, up to three attempts; if repair fails the API returns a 502 with the failing stage instead of a fabricated report. After validation, every extracted value (hostname, IPs, user, timestamp, evidence, indicators) must appear verbatim in the alert, otherwise it is removed and listed as a limitation.

RAG. Each playbook has a stable ID (PB-001..PB-019), title, version, category and provenance, and is explicitly labelled fictional. Documents are cleaned (the repeated fictional banner is not indexed) and split per markdown section with the title and heading prefixed. Retrieval scores chunks with BM25 and all-MiniLM-L6-v2 cosine similarity, fuses them as alpha*cos + (1-alpha)*bm25/(bm25+8), ranks documents by their best chunk and abstains when the top score is below a dev-tuned threshold. The report may only cite retrieved IDs; violations trigger repair and then a validation error.

Agent. Tools: search_playbooks, calculate_severity, lookup_incident_history - all read-only, each with a Pydantic argument model (extra fields forbidden, bounded integers, hostname regex). Native OpenAI tool calling is used when available; otherwise a documented JSON action protocol is used and labelled as such in the trace. The loop stops on a final answer, 6 model calls, 150 s, 3 consecutive invalid actions or tool errors, or a transport error; duplicate calls are refused. If the agent skips retrieval or computes severity with factors that differ from the grounded extraction, the orchestrator runs the tool itself and records an enforced step. Alert text, excerpts and tool results are wrapped in data tags and treated as untrusted.

Severity. score = tier(1-4) + confidence(1-3) + 2*impact(0-3) + 2 if privileged; bands 0-4 Informational, 5-6 Low, 7-8 Medium, 9-10 High, 11+ Critical. The tier comes from the catalogue, never from the model; unknown

assets default to medium and are flagged.

5. Dataset construction and methodology

The evaluation set contains 32 manually written queries (16 dev, 16 test), each with relevant document IDs, expected incident type, expected severity where determinable (with the factors), expected abstention and a note justifying the label. Scenario mix: ambiguous 1, answerable 17, false positive 2, insufficient evidence 2, out of domain 4, policy question 3, similar different playbook 3. Automated checks verify that every relevant ID exists, that severity labels agree with PB-002 given their factors, and that no query shares a 6-word sequence with the corpus (to avoid trivially copied wording). The corpus, alerts, assets and history are synthetic; IPs come from RFC 5737 documentation ranges.

6. Evaluation metrics and baseline

- Recall@3 = |relevant in top 3| / |relevant|; MRR@5 = 1/rank of first relevant document in top 5 (else 0); Precision@3 and Hit@1 as secondary metrics. All averaged over **answerable** queries only.
- Abstention accuracy over **all** queries: correct iff the retriever abstains exactly when no relevant document exists.
- **Baseline A**: BM25 over fixed 120-word chunks - the naive first implementation.
- **Candidates**: BM25 with section chunks; dense LSA (TF-IDF + SVD, seed 42); dense MiniLM; hybrids with alpha in {0.3, 0.5, 0.7}.
- **Protocol**: thresholds tuned on dev only; selection rule fixed in advance: max dev mean(Recall@3, MRR@5); tie-break dev abstention accuracy; then list order. Test results are reported once for the baseline and the selected configuration (all others are shown for transparency but were not used for any decision).

7. Experimental results

Table 1. Baseline versus selected configuration on the held-out test split (16 queries, 14 answerable).

Held-out TEST	Recall@3	MRR@5	P@3	Hit@1	Abstention acc.
A: bm25-fixed	0.679	0.696	0.238	0.643	0.875
B: hybrid-sbert-section-a0.3 (selected on dev)	0.750	0.729	0.262	0.643	0.875

Per-query reciprocal rank on test versus the baseline: 2 wins, 1 loss(es), 11 ties. On dev the selected configuration reached Recall@3 0.964 / MRR@5 0.952 versus 0.929 / 0.881 for the baseline.

Table 2. All configurations (selection used the dev columns only).

Configuration	chunks	dev R@3	dev MRR@5	dev abst.	test R@3	test MRR@5	test abst.
bm25-fixed	40	0.929	0.881	1.000	0.679	0.696	0.875
bm25-section	85	0.964	0.917	1.000	0.750	0.750	0.875
dense-lsa-fixed	40	0.786	0.857	0.875	0.750	0.750	0.875
dense-lsa-section	85	0.893	0.875	0.875	0.750	0.679	0.875
hybrid-lsa-section-a0.3	85	0.893	0.911	1.000	0.750	0.714	0.812
hybrid-lsa-section-a0.5	85	0.893	0.911	0.875	0.750	0.714	0.875
hybrid-lsa-section-a0.7	85	0.893	0.911	0.875	0.750	0.679	0.875
dense-sbert-fixed	40	0.929	0.893	1.000	0.964	0.845	1.000
dense-sbert-section	85	0.857	0.800	1.000	0.821	0.854	1.000
hybrid-sbert-section-a0.3	85	0.964	0.952	1.000	0.750	0.729	0.875
hybrid-sbert-section-a0.5	85	0.929	0.952	1.000	0.821	0.762	0.938
hybrid-sbert-section-a0.7	85	0.929	0.952	1.000	0.821	0.809	1.000

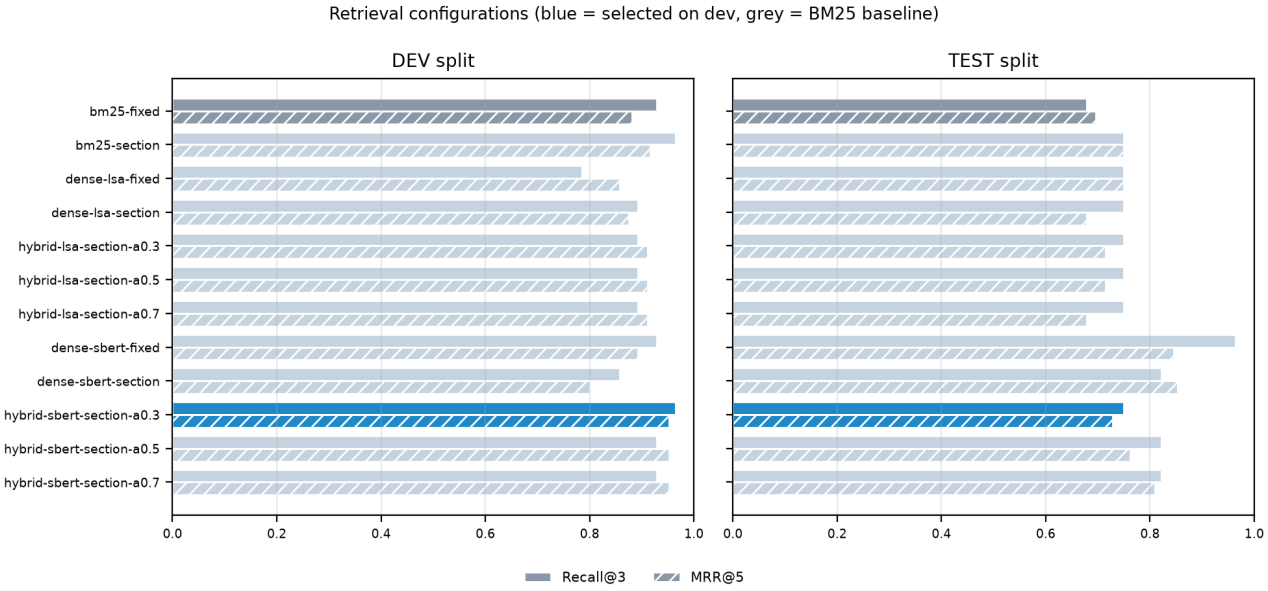


Figure 2. Recall@3 and MRR@5 per configuration on dev and test (generated from eval/results/retrieval_runs.json).

End-to-end application evaluation (Gemma)

Every labelled alert is sent through the full live pipeline. The first run (r0) found that Gemma hedged alert confidence to 'medium' and under-rated impact, so the pipeline was improved in three rounds (docs/EXPERIMENT_LOG.md). Before each round a new 16-alert holdout set was written and labelled, giving every change one measurement on data it was not designed from (Table 4). The original test split was inspected after r0 and is therefore no longer blind.

Table 4. Each round measured on the holdout set written before it.

Round	Change	Blind set	Severity acc.	Relevant citation
r1	explicit confidence/impact definitions; insufficient-evidence search	holdout v2 (16)	0.462 -> 0.615	0.750 -> 0.812
r2	evidence kind -> deterministic confidence; retry search after abstention	holdout v3 (16)	0.923 -> 0.923	0.812 -> 0.938
r3	yes/no outcome-confirmed flag; label-only last-resort search	holdout v4 (16)	0.615 -> 0.846	1.000 -> 1.000

Table 5. Test split (12 alerts, 7 with a severity label), first run vs. final configuration (not blind after r0).

Metric (test split)	r0	final
valid structured output rate	1.000	1.000
incident type accuracy	1.000	1.000
severity accuracy	0.571	1.000
citation validity rate	1.000	1.000
relevant citation rate	0.917	1.000
undetermined flagged rate	1.000	1.000
native tool mode rate	1.000	1.000

Final run 2026-10-17T07:27:11+00:00 (r3_outcome_flag_label_fallback). A repeat run of r2 gave identical severities and incident types, so round differences are not sampling noise. r3 lost one severity and one citation case on the seen holdout v3; pooled over all sets correct severities rose from 40/51 to 43/51.

8. Success cases and failure analysis

Successes. Heading-aware chunking alone raised BM25 test Recall@3 from 0.679 to 0.750, because section titles such as 'Recognising the pattern' and 'Indicators' concentrate the discriminative vocabulary. The hybrid recovered the insufficient-evidence procedure for a vague alert (Q24) that the baseline ranked fourth. With a mocked LLM the pipeline tests show that fabricated citations are repaired or rejected, invented hostnames are removed, and injected 'run_shell' requests are refused.

Table 3. Remaining test failures of the selected configuration.

Query	Scenario	Relevant	Top-5 (selected)	Cause (manual analysis)
Q02	answerable	PB-004	PB-018, PB-013, PB-005, PB-006, PB-004	hostname token LAB-TEST-07 matches the asset catalogue (PB-018), outranking PB-004
Q04	answerable	PB-005	PB-016, PB-001, PB-018, PB-004, PB-015	no lexical overlap (city names) and weak dense score -> false abstention
Q20	similar different playbook	PB-017|PB-005	PB-017, PB-004, PB-012, PB-013, PB-003	second relevant document (PB-005) ranked below 3
Q26	policy question	PB-016	PB-001, PB-018, PB-005, PB-011, PB-006	vocabulary mismatch: 'cut off the network' vs 'isolate'
Q30	out of domain	none (abstain)	PB-005, PB-015, PB-017, PB-018, PB-001	generic business words score just above the abstention threshold

Honest observations. (1) Metrics dropped from dev to test for every configuration, so the dev split was easier; with 14 answerable test queries one query moves Recall@3 by about 0.07. (2) dense-sbert-fixed was the best configuration on test but was not chosen, because choosing it now would mean tuning on the test set. (3) The LSA variant was weak on this tiny corpus. Future work, to be decided on new dev data: stripping hostnames/IPs from retrieval queries, query expansion with synonyms (isolate/disconnect), and a larger, independently written dev set.

9. Security considerations

- No shell, file-system, network-scanning or write tools exist; tool names are allow-listed and arguments are schema-validated (tests include path traversal and SQL-injection strings).
- Prompt injection: untrusted content is tagged as data, injection phrases are detected and force human review; architectural limits make tool misuse harmless.
- Citations must be retrieved IDs; severity is computed by code; ungrounded values are removed.
- Secrets come only from environment variables (.env is git-ignored) and are never logged or returned; document IDs are matched against ^PB-\d{3}$ and served from memory; strict CSP and textContent rendering in the UI.

10. Limitations and future work

- Synthetic, small corpus and dataset; results do not transfer to real SOC telemetry.
- End-to-end sets are small (12-16 alerts); Gemma still rates impact 'suspected' on some alerts that show confirmed execution, and evidence-kind calls are unstable on borderline alerts.
- Literal grounding drops correct paraphrases; pattern-based injection detection can be evaded.
- Single global abstention threshold; no authentication or multi-user isolation.
- Future: multi-alert correlation, analyst feedback loop, larger evaluation set, query rewriting, LLM-as-judge for report quality with human calibration.

Reproducibility

Command: `python -m eval.run_retrieval_eval`; python 3.13.12; LSA seed 42; BM25 k1=1.5, b=0.75; embedding model sentence-transformers/all-MiniLM-L6-v2. Per-example results: eval/results/per_example.csv. Tests: `pytest` (offline, mocked LLM).

References: Nat-D/kmitl-llm-app-demo (course reference, ideas only); Robertson & Zaragoza (2009) BM25; Reimers & Gurevych (2019) Sentence-BERT; Deerwester et al. (1990) LSA; RFC 5737.

Executive summary (Thai)

SentinelAI คือผู้ช่วยวิเคราะห์เหตุการณ์ด้านความปลอดภัย (SOC Triage Copilot) ที่ใช้โมเดลภาษา gemma-4-E4B-it ผ่าน proxy ของรายวิชา เพื่อช่วยนักวิเคราะห์ SOC ระดับเริ่มต้นคัดกรองแจ้งเตือนความปลอดภัย
ข้อมูลทั้งหมดเป็นข้อมูลสังเคราะห์ ขององค์กรสมมติ และระบบเป็นเพียงผู้ช่วย
ไม่ได้ดำเนินการตอบสนองเหตุการณ์เองโดยอัตโนมัติ

การทำงานหลัก

- สกัดข้อมูลจากแจ้งเตือนเป็นโครงสร้างที่ตรวจสอบด้วย Pydantic และตัดค่าที่ไม่ปรากฏในข้อความจริงทิ้ง (ป้องกันการแต่งข้อมูล)
- Agent เลือกใช้เครื่องมือที่อนุญาตเท่านั้น (ค้นหา playbook, คำนวณความรุนแรง, ค้นประวัติเหตุการณ์) ภายใต้ขีดจำกัดจำนวนชั้นและเวลา
- คำนวณระดับความรุนแรงด้วยตารางคะแนนแบบกำหนดแน่นอน (PB-002) ซึ่งโมเดลไม่สามารถแก้ไขได้
- สร้างรายงานพร้อมอ้างอิงเอกสารที่ถูกค้นคืนจริง หากอ้างอิงไม่ถูกต้องระบบจะให้แก้ไข และหากยังผิดจะคืนข้อผิดพลาดแทนการสร้างรายงานปลอม

ผลการประเมินที่วัดจริง (ชุดทดสอบที่แยกไว้ 16 คำถาม, ตอบได้ 14 ข้อ)

- Baseline BM25 แบบ chunk คงที่: Recall@3 = 0.679, MRR@5 = 0.696
- ค่าที่เลือกจากชุด dev (Hybrid BM25 + MiniLM, alpha = 0.3, chunk ตามหัวข้อ): Recall@3 = 0.750, MRR@5 = 0.729
- ดีขึ้นเล็กน้อย (ชนะ 2 แพ้ 1 เสมอ 11 คำถาม) แต่ชุดข้อมูลมีขนาดเล็ก จึงยังไม่สามารถสรุปเชิงสถิติได้

การประเมินแบบ end-to-end กับ Gemma จริง (gemma-4-E4B-it, temperature 0)

- รอบแรก (r0) ความแม่นยำของระดับความรุนแรงบนชุดทดสอบอยู่ที่ 0.571 เพราะโมเดลให้ alert confidence เป็น "medium" เกือบทุกเคส
- ปรับปรุง 3 รอบ ทุกรอบเขียนชุด holdout ใหม่ 16 เคสก่อนแก้ เพื่อวัดผลกับข้อมูลที่ระบบไม่เคยเห็น
- ผลบนชุดที่ไม่เคยเห็น (severity): r1 0.462 □ 0.615, r2 0.923 □ 0.923, r3 0.615 □ 0.846
- ผลบนชุดที่ไม่เคยเห็น (อ้างอิง playbook ที่เกี่ยวข้อง): r2 0.812 □ 0.938
- เวอร์ชันสุดท้าย (r3) บนชุดทดสอบเดิม: output ถูก schema, ประเภทเหตุการณ์, ความรุนแรง และการอ้างอิง = 1.000 ทั้งหมด แต่ชุดนี้เคยถูกดูเคสที่ผิดมาก่อนแก้ จึงไม่ใช่ตัวเลขแบบ blind
- รายละเอียดทุกรอบอยู่ใน docs/EXPERIMENT_LOG.md

ข้อจำกัด: ข้อมูลสังเคราะห์ขนาดเล็ก, ชุดทดสอบเล็ก, การตรวจจับ prompt injection เป็นแบบ pattern,
ไม่มีระบบยืนยันตัวตน จึงควรรันในเครื่องเท่านั้น